

DataFrame of Mind

Data pipelines with Polars and PySpark · Data Minded Academy



Contents

- 01 Data and schemas

- 02 The engines: pandas, DuckDB, Polars

- 03 Expressing a query

- 04 How the engine executes it

- 05 Choosing an engine

- 06 PySpark



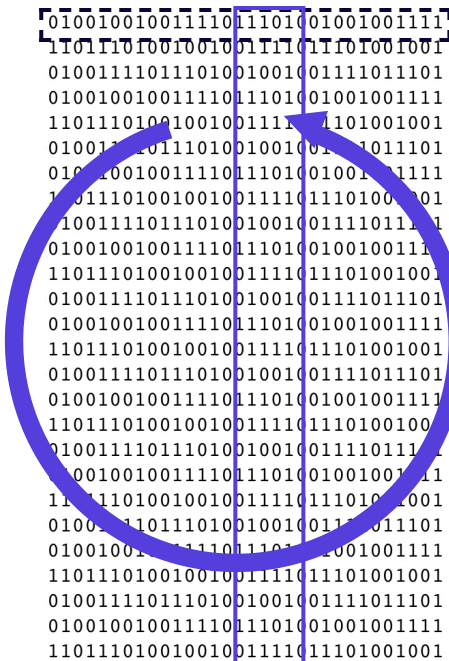
Data and schemas



Operational vs. Analytical Data

Operational ⚡

- Detailed events
- Fast and complete (ACID)
- Reads and writes **one record**, all of its fields



Analytical 🔍

- Descriptive and predictive
- Slow and selective
- Reads **one field** across every record

Operations capture reality event by event, analytics finds the patterns, and the patterns change how the next event is served.



Exercise time: operational vs. analytical data

1-operational-vs-analytic/

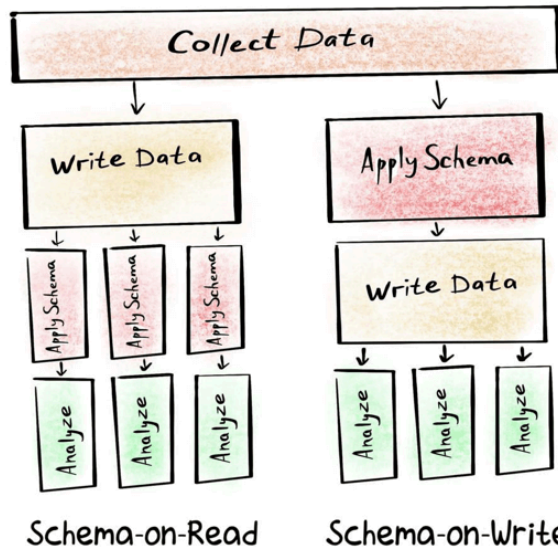


Data is valuable when it has **structure**



⚡ faster writes

💰 resource-intensive
(storage, processing)



Schema-on-Read

Schema-on-Write

@luminousmen.com



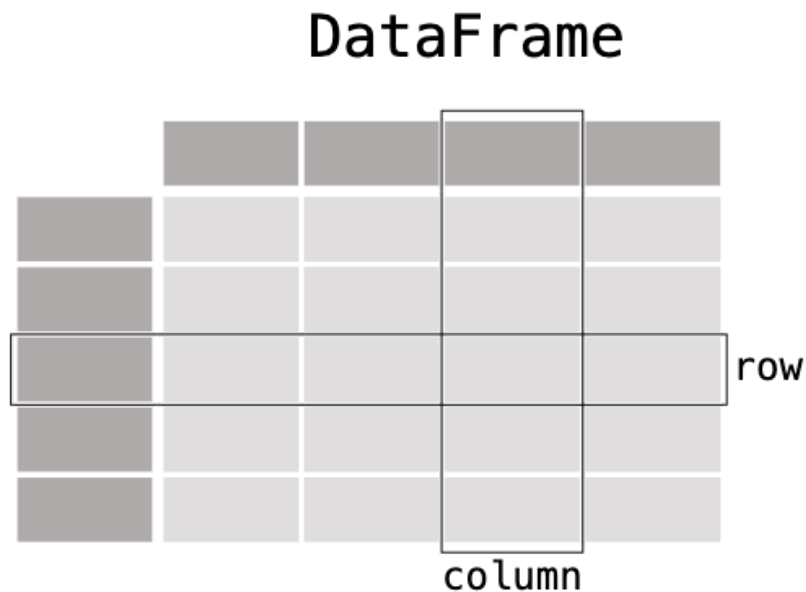
🐢 slower writes
🦉 resource-efficient



It's not a question of **whether** you apply a schema, it's a question of **when**.



A dataset with a schema is usually a **DataFrame**



Extracting and loading data: a wide range of possibilities

Operational ⚡ (extract)

- CSV / TSV
- Avro
- TXT
- RDBMS (Postgres, MySQL, SQLite, ...)

**Avro****TXT****SQLite**

Analytical 🔍 (load)

- Parquet (Delta, Iceberg)
- ORC
- Feather / Arrow IPC
- DuckDB file format

**Parquet**Apache
orc

Bring in the Animals 🐼 🦆 🐻



Query engines power the **transformations** in ETL



Extract from many sources, transform in the engine, load into analytical storage.



pandas, DuckDB and Polars compared

	 pandas	 DuckDB	 Polars
Born 	2008 	2019 	2020 
Written in 	Python (C, Fortran)	C++	Rust
GitHub stars 	47.8K	35.8K	37.3K
Loved by 	Data scientists 	Data analysts 	Data engineers 



Polars is a side project that got out of hand 🙌

- Main author: Ritchie Vink 🇳🇱, structural engineer by education
- Started development during the COVID-19 pandemic 🤔
- Full-time on Polars since July 2023, when he founded Polars Inc. to fund the work
- The company sells enterprise support and Polars Cloud
- One of the fastest growing data manipulation tools in the Python ecosystem



Demo time: a dirty, dirty CSV

`demo-reading-data/` and `2-csv-from-hell/`



Querying and Transforming Data



Imperative, declarative and functional styles

Imperative

```
affordable_cars = []  
for car in cars:  
    if car.price <= 30_000:  
        affordable_cars.append(car)
```

Step by step, like a recipe.

Declarative

```
SELECT brand, price FROM cars  
WHERE price <= 30000
```

Describe what you want, let an optimised engine work out how.

Functional

```
cars = pl.read_csv("cars.csv")  
affordable = cars.filter(  
    pl.col("price") <= 30_000  
)
```

Chain functions into a pipeline, no duplication, no mutation.



Relational algebra, the cornerstone of RDBMS

$$\begin{array}{ll}
 \Pi_{a_1, \dots, a_n}(A) & A \bowtie_{\theta} B \\
 \sigma_c(A) & A \bowtie_c B \\
 \rho_{a/b}(A) & A \bowtie B \\
 \rho_{B(a_1, \dots, a_n)}(A) & A \times B \quad A \ltimes B \\
 A \cup B & A \bowtie B \quad A \bowtie B \\
 A \cap B & A \bowtie B \\
 A - B & A \bowtie B \\
 A \times B & A \div B
 \end{array}$$

Projection, selection, rename, set operations and the whole family of joins.



Project, filter, rename, union, join

SQL

```
SELECT
  price as car_price
FROM
  (
    SELECT * FROM old_cars
    UNION
    SELECT * FROM new_cars
  )
WHERE car_price > 30000
```

Polars

```
import polars as pl

df = (
    pl.concat(
        pl.read_csv("old_cars.csv"),
        pl.read_csv("new_cars.csv"),
    )
    .select(
        pl.col("price").alias("car_price")
    )
    .filter(pl.col("car_price") > 30_000)
)
```



To SQL or not to SQL?

SQL wins on

- Readability
- Getting stuff done
- Easy to learn, everyone speaks it



A DataFrame API wins on

- Abstraction (functions, modules)
- Control flow (if, for, while)
- Testing
- Ecosystem (linting, typing, packaging)



Beyond relational algebra: contexts and expressions

Polars adds its own DSL on top of the relational engine. An **expression** is a tree of operations describing how to build one or more Series. Expressions are always evaluated inside a **context**: `select`, `with_columns`, `filter` and `group_by`.

```
df = pl.DataFrame({
    "integer": [1, 2, 3],
    "date": [datetime(2024, 1, 1), datetime(2024, 1, 2), datetime(2024, 1, 3)],
    "float": [4.0, 5.0, 6.0],
    "text": ["a", "b", "c"],
})

query = df.with_columns(                                # context
    (pl.col("float") * pl.col("integer")).alias("int*float") # expression
).filter(                                              # context
    pl.col("date") >= datetime(2024, 1, 2)           # expression
).select(                                             # context
    pl.all()                                          # expression
)
```



Polars comes with a big bag of **batteries** included

Column selectors

```
from polars import selectors as cs

df.select(cs.contains("a"))
df.select(cs.numeric())
```

Meta-queries over the schema,
instead of hard-coded column lists.

Type namespaces

```
df.with_columns(
    pl.col("baz").str.to_uppercase()
)
df.with_columns(
    pl.col("ts").dt.year()
)
```

Type-specific functions live in

`.str`, `.dt`, `.list` and `.struct`.

Testing helpers

```
from polars.testing import (
    assert_frame_equal)

assert_frame_equal(df1, df2)
# AssertionError: columns
# ['foo', 'bar', 'baz'] in left
# DataFrame, but not in right
```

Frame and series comparisons that
fail with a readable message.



Exercise time: relational algebra

3-basic-transforms/



Windowing and aggregations

Window: one row in, one row out

```
import polars as pl

df = (
    pl.read_csv("cars.csv")
    .with_columns(
        pl.col("price")
        .min()
        .over("model_type")
        .alias("price_without_options"),
    )
)
```

Calculate a value over a group and add it to every record of that group.

Aggregation: one group, one row

```
import polars as pl

df = (
    pl.read_csv("cars.csv")
    .group_by("model_type")
    .agg(
        pl.col("price")
        .min()
        .alias("price_without_options"),
    )
)
```

Calculate a value over a group and return one record per group.

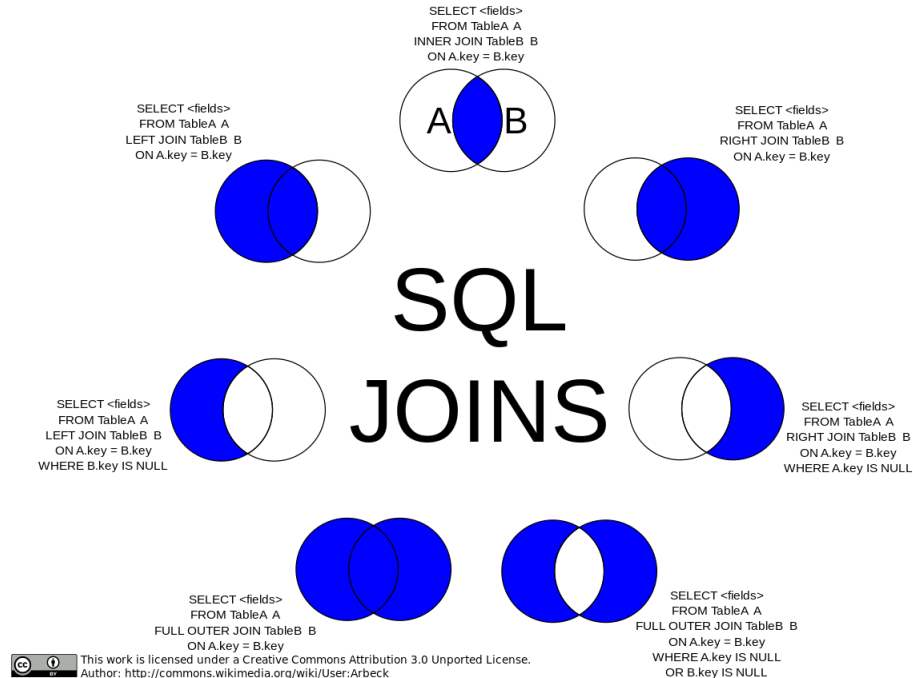


Exercise time: windowing and aggregations

4-window-aggregations/



The standard join types



Polars supports all **standard** join operations

```
DataFrame.join(  
    other: DataFrame,  
    on: str | Expr | Sequence[str | Expr] | None = None,  
    how: JoinStrategy = 'inner',  
    *,  
    left_on: str | Expr | Sequence[str | Expr] | None = None,  
    right_on: str | Expr | Sequence[str | Expr] | None = None,  
    suffix: str = '_right',  
    validate: JoinValidation = 'm:m',  
    join_nulls: bool = False,  
    coalesce: bool | None = None,  
) -> DataFrame
```

validate is worth remembering: it turns a silent row explosion into an error.



... and even non-standard joins

```
DataFrame.join_asof(  
  other: DataFrame,  
  left_on: str | None | Expr = None,  
  right_on: str | None | Expr = None,  
  on: str | None | Expr = None,  
  by_left: str | Sequence[str] | None = None,  
  by_right: str | Sequence[str] | None = None,  
  by: str | Sequence[str] | None = None,  
  strategy: AsofJoinStrategy = 'backward',  
  suffix: str = '_right',  
  tolerance: str | int | float | timedelta | None = None,  
  allow_parallel: bool = True,  
  force_parallel: bool = False,  
) -> DataFrame
```

Match on the nearest key rather than an exact one. Useful for time series: sensor readings joined to the most recent configuration change.



Exercise time: joins

5-joins/

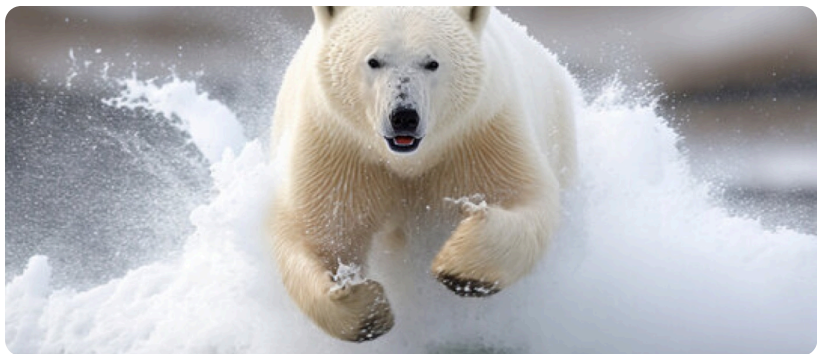


How the engine executes it



Lazy vs. **eager** evaluation

Eager: `read_csv`



Every step runs the moment you write it. Fine for exploration, wasteful for pipelines.

Lazy: `scan_csv`



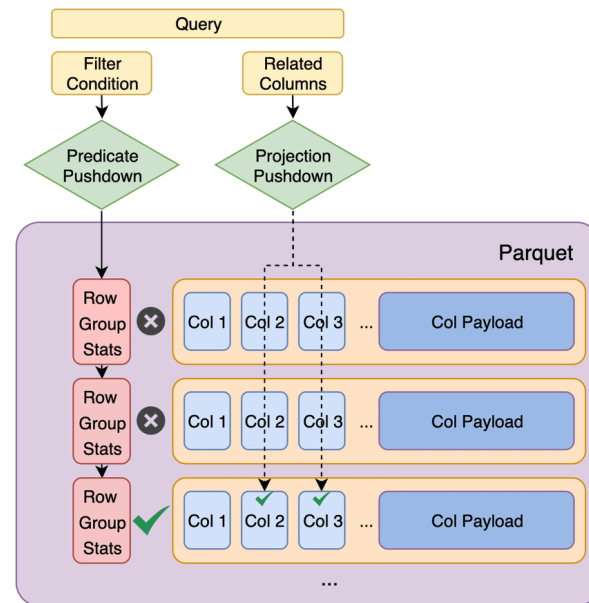
Nothing runs until `.collect()`. The optimiser sees the whole query and rewrites it.



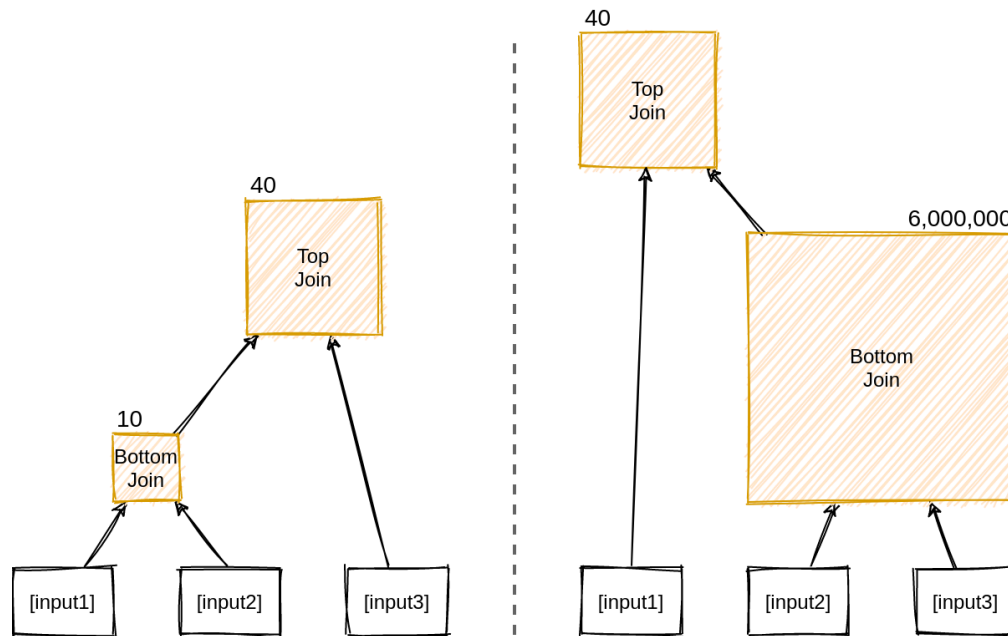
Some optimisations: pushdown

- **Predicate pushdown:** filter rows as early as possible
- **Projection pushdown:** drop columns as early as possible
- **Query pushdown:** push filters, joins and aggregations into the data source

Typically applied while reading the data, so the rows never enter memory at all.



Some optimisations: join reordering



Same three inputs, same result, 6 million intermediate rows of difference.



Polars streaming: when you are resource-constrained

Instead of processing the data all at once, Polars can execute the query in batches, which lets you process datasets larger than memory.

```
q1 = (  
    pl.scan_csv("car.csv")  
    .filter(pl.col("price") < 30_000)  
    .group_by("brand")  
    .agg(pl.col("max_speed").max())  
)  
df = q1.collect(streaming=True)
```

Supported operations

filter, slice, head, tail, with_columns,
select, group_by, join, unique, sort, explode,
melt, scan_csv, scan_parquet, scan_ipc



A bit unread is a bit less to process



Pushdown is the engine skipping bytes for you. It can only skip what the layout allows, so partition along the column you filter on first.



Demo: hive-partitioned vs. plain Parquet



When SQL doesn't cut it: user defined functions

map_rows

```
df.map_rows(  
    lambda t: (t[0] * 2, t[1] * 3)  
)
```

Every row as a tuple. Column names are lost, and it is the slowest of the three.

map_batches

```
pl.col("features").map_batches(  
    lambda s: model.forward(  
        s.to_numpy()  
    )
```

The whole Series at once, so NumPy and PyTorch run once per column.

map_elements

```
pl.col("a").map_elements(  
    lambda x: x * 2,  
    return_dtype=pl.Int64,  
)
```

One value at a time. Pass `return_dtype` or Polars infers it from the first result.

Every UDF drops out of the optimised engine and back into the Python interpreter, so check for a builtin first.



Exercise time: UDFs

6-udf/



Big Data?



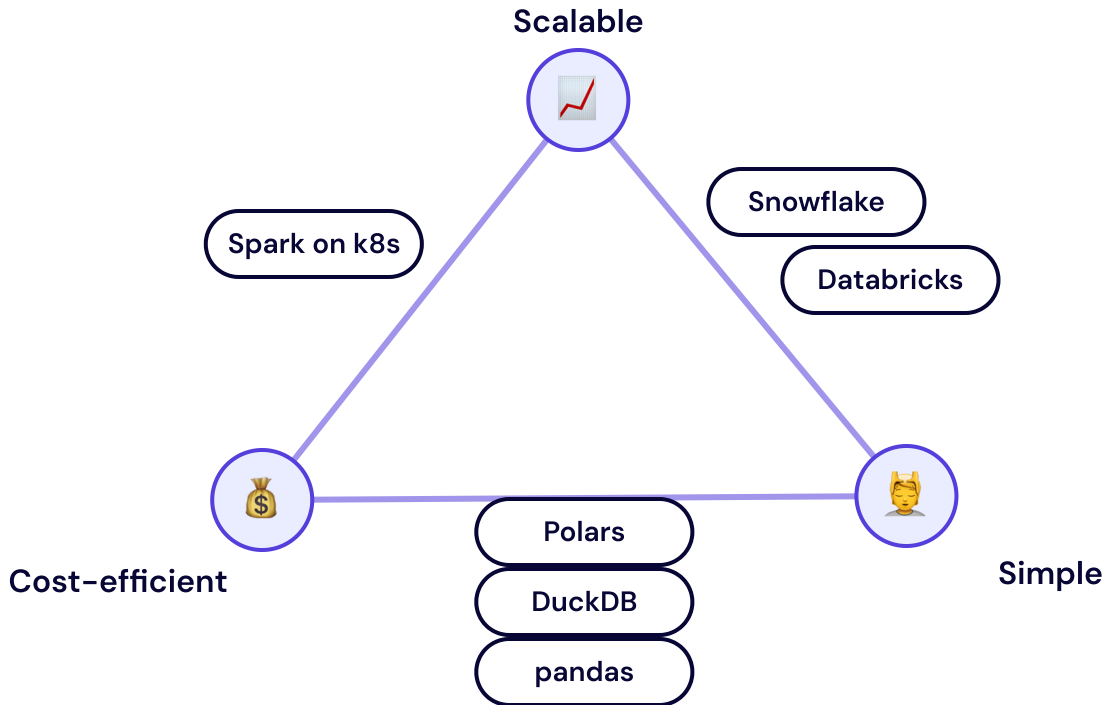
The data processing landscape over the years



- 📈 First wave optimised for **scale**: commodity machines, open source, structured and semi-structured data
- 🏠 Second wave optimised for **simplicity**: managed platforms hid the cluster
- 💰 Third wave optimised for **cost**: Kubernetes brought the cluster back, cheaper

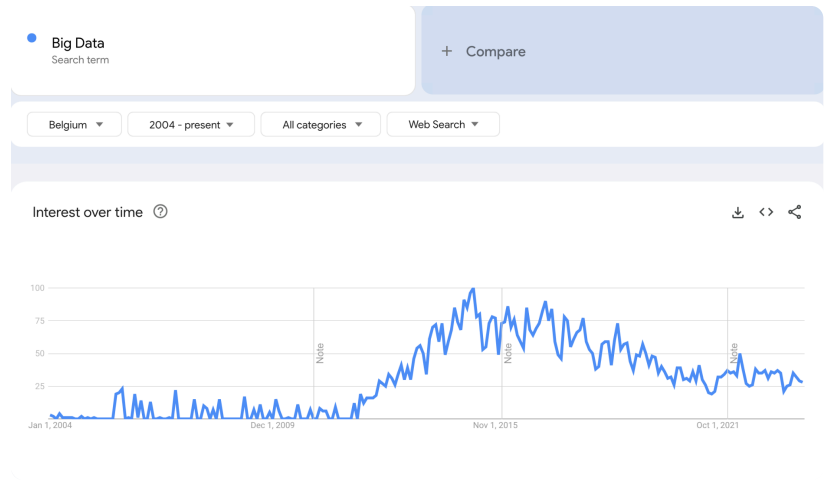


Emerging single node processing technologies



Interest in big data, and what one machine **now** holds

Interest in 'big data' is past its peak



One machine now holds 12 TB of RAM

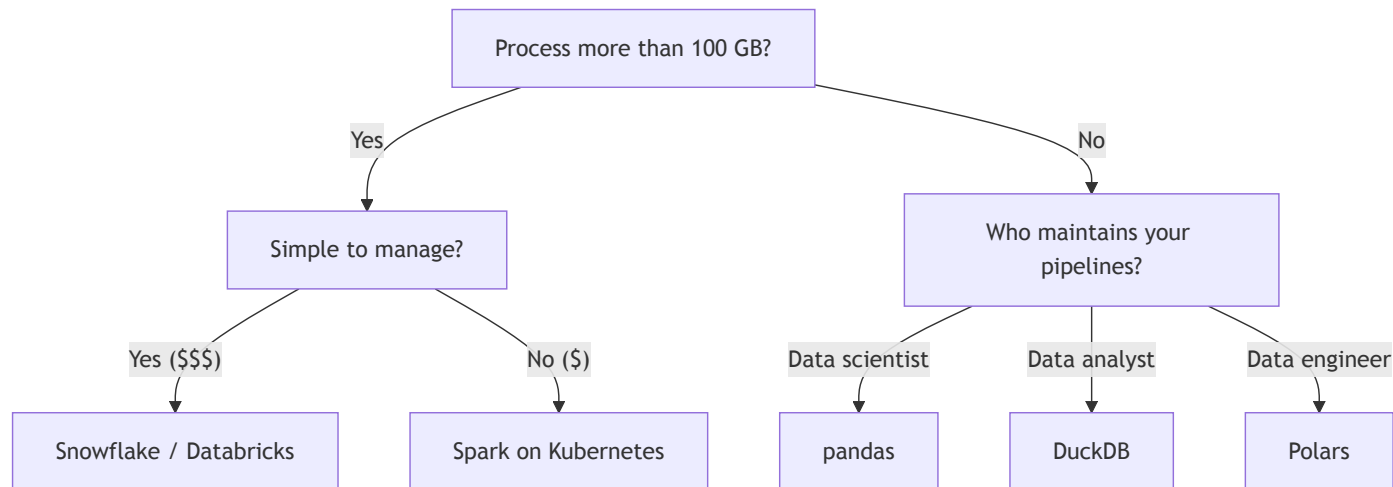
Instance name ▾	On-Demand hourly rate ▾	vCPU ▾	Memory ▾
u-12tb1.112xlarge	\$109.20	448	12288 GiB
u-6tb1.112xlarge	\$54.60	448	6144 GiB
u-6tb1.56xlarge	\$46.40391	224	6144 GiB
x2iedn.32xlarge	\$26.676	128	4096 GiB
x2iedn.metal	\$26.676	128	4096 GiB
x1e.32xlarge	\$26.688	128	3904 GiB
u-3tb1.56xlarge	\$27.30	224	3072 GiB
x2iedn.24xlarge	\$20.007	96	3072 GiB



You can't please all of the people all the time,
luckily there is an **escape hatch**



When do I use **what**?



Apache Spark



The same query in PySpark and Polars

PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.window import Window
import pyspark.sql.functions as sf

spark = SparkSession.builder.getOrCreate()

df = (
    spark.read.csv("measurements.csv")
    .withColumn("year", sf.year("timestamp"))
    .withColumn("max_weight_per_year",
                sf.max("weight")
                  .over(Window.partitionBy("year")))
    )

df.collect()
```

Polars

```
import polars as pl

df = (
    pl.scan_csv("measurements.csv")
    .with_columns(
        pl.col("timestamp").dt.year()
        .alias("year")
    )
    .with_columns(
        pl.col("weight").max().over("year")
        .alias("max_weight_per_year")
    )
)

df.collect()
```

Both express the same operations, and both are lazy. The rest of this section covers what changes when the work runs on a cluster instead of in one process.



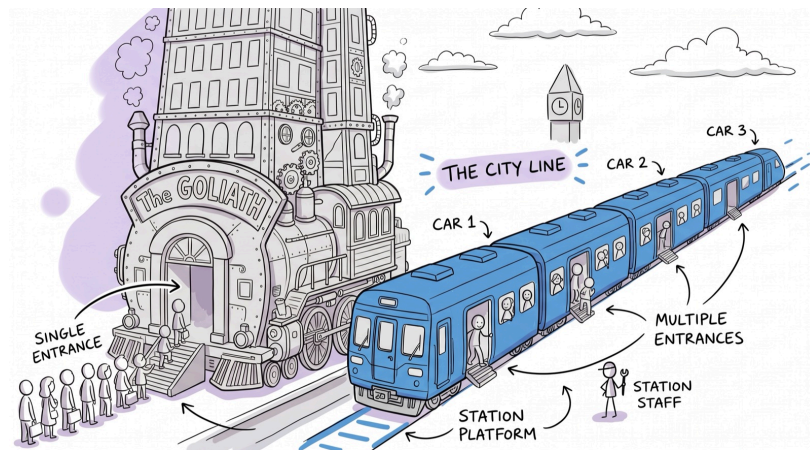
Large jobs are limited by I/O, and a cluster adds bandwidth

One machine

- One network card, one set of disks: a few GB/s at best
- 10 TB at 1 GB/s: three hours before any computation starts
- The CPUs idle, waiting for bytes

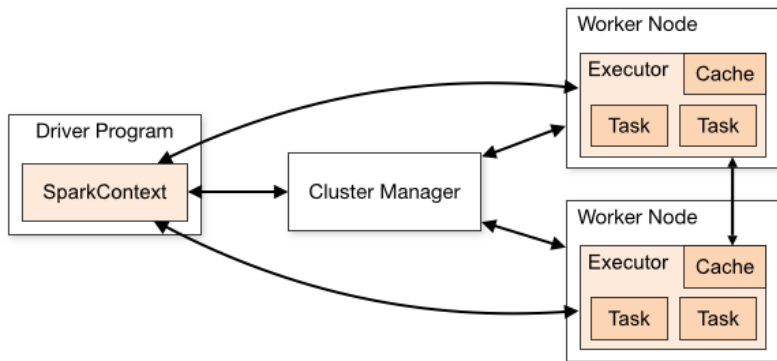
A cluster

- Every executor reads its own slice, so bandwidth grows with the machine count
- 10 TB across 60 executors: minutes, if the object store keeps up
- Memory scales the same way



A submitted job runs on a driver and its executors

Scaling out means adding machines rather than buying a bigger one. This is the machinery that makes that possible.



Your program runs in the driver. The cluster manager allocates executors on worker nodes, and the driver ships the code and the tasks to them.

```
$ ./bin/spark-submit \
  --master k8s://https://<host>:<port> \
  --deploy-mode cluster \
  --name spark-pi \
  --conf spark.executor.instances=5 \
  --conf spark.kubernetes.container.image=<image> \
  local:///path/to/examples.jar
```

`spark-submit` sets `SPARK_HOME` and `JAVA_HOME` and hands your program to the launcher.

Note the arrow between the workers: executors talk to each other, not only to the driver. The next slide is about that arrow.



Wide transformations **shuffle** data across the network

Narrow: no data moves

`select`, `filter`, `withColumn`, `union`

Each executor works on the partitions it already holds.
Spark chains these into one stage.

Wide: rows meet on one executor

`groupBy`, `join`, `distinct`, `sort`, `repartition`

All rows with the same key have to end up in the same place, so Spark writes shuffle files and moves them over the network. That ends a stage and starts the next.

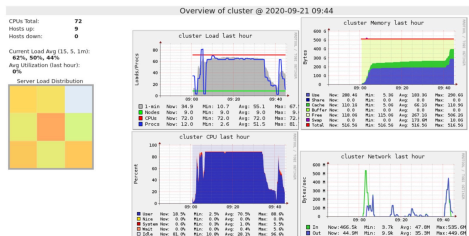
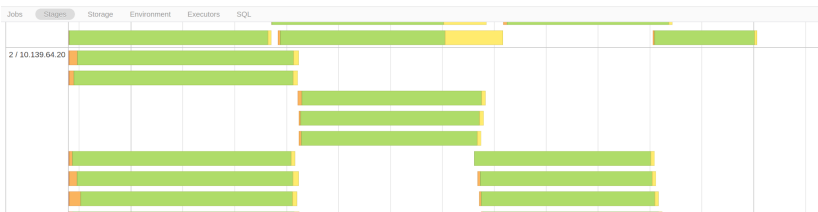
```
df.filter(sf.col("age") > 18).groupBy("country").count().explain()
# AdaptiveSparkPlan isFinalPlan=false                                (abridged)
# +- HashAggregate(keys=[country], functions=[count(1)])
#   +- Exchange hashpartitioning(country, 200)      <- the shuffle
#      +- HashAggregate(keys=[country], [partial_count(1)])
#         +- Filter (age > 18)
```

No Polars equivalent: in one process a `group_by` is a hash table, not a network transfer.

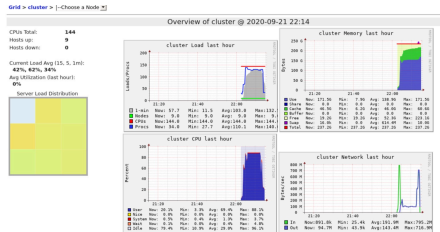


Partition count decides how much of the cluster stays busy

72 vCPUs, idle at the tail



Partitions matched to cores



The 200 in that `Exchange` is the default of `spark.sql.shuffle.partitions`, applied to every shuffle whatever your data or cluster size. Make it a multiple of the cluster cores. Since Spark 3.2 adaptive execution coalesces them at runtime, so treat 200 as a starting point.



Transformations are **lazy**, actions trigger execution

```
import pyspark.sql.functions as sf

df1 = spark.range(3)
df2 = df1.withColumn("foo", sf.lit("bar"))
df3 = df2.withColumn("bar", sf.col("id") < 2)
df4 = df3.filter(sf.col("id") != 1)
df5 = df4.select("foo", "bar", "id",
                 (sf.col("id") + 2).alias("plus2"))
df6 = df5.drop("id")
df7 = df6.withColumnRenamed("bar", "lessthan2")

df7.printSchema()    # free: bookkeeping only
print(df7.count())    # action: the plan runs
df7.show()           # action
```

Polars taught you this with `scan_csv` and `collect`.

Spark works the same way: transformations build a plan, actions run it.

Two things are different once the plan runs on a cluster:

- ⚠ Errors surface at the **action**, not at the line that caused them, and they arrive as a Java stack trace from an executor.
- `DataFrame.explain()` shows the physical plan, including every `Exchange`.



Chaining drops the intermediate names, **transform** makes steps testable

Chain the calls

```
df = (
    spark.range(3)
    .withColumn("foo", sf.lit("bar"))
    .withColumn("bar", sf.col("id") < 2)
    .filter(sf.col("id") != 1)
    .select("foo", "bar", "id",
            (sf.col("id") + 2).alias("plus2"))
    .drop("id")
    .withColumnRenamed("bar", "lessthan2")
)
```

No `df1` to `df7` to keep straight, and no filtering `df3` when you meant `df4`.

Name the steps with transform

```
def add_flags(df: DataFrame) -> DataFrame:
    return (df
            .withColumn("foo", sf.lit("bar"))
            .withColumn("bar", sf.col("id") < 2))

def drop_middle(df: DataFrame) -> DataFrame:
    return df.filter(sf.col("id") != 1)

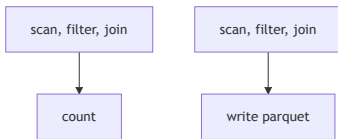
df = (spark.range(3)
      .transform(add_flags)
      .transform(drop_middle))
```

A `DataFrame -> DataFrame` function is testable on three rows, no cluster involved. **transform** is PySpark's pipe; Polars spells it `.pipe()`.

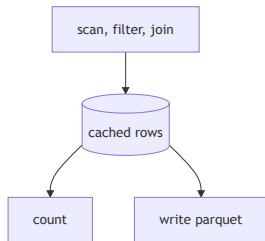


A DataFrame used twice is computed twice, unless you **cache** it

Two actions, two full recomputations



Cached: computed once, read twice



```
enriched = (spark.read.parquet("events")
    .filter(sf.col("year") == 2024)
    .join(dim_user, "user_id")
    .cache())           # lazy too: the first action fills it
enriched.count()       # computes, then keeps the rows
enriched.write.parquet("out") # reads the cache instead
```

Only worth it when one DataFrame feeds more than one action. Call `unpersist()` when the branch is done.



A UDF runs Python per row, **outside** Spark's optimiser

```
import pyspark.sql.functions as sf
from pyspark.sql.types import IntegerType

def square(x):
    return x**2

square_as_udf = sf.udf(
    square, returnType=IntegerType())
df.withColumn("squared", square_as_udf("id"))
```

You must declare the return type: the optimiser cannot infer it from Python.

In Polars a UDF drops from Rust into Python in the same process. In Spark, every row is serialised out of the JVM into a separate Python worker and back again.

That round trip is why `pandas_udf` exists: it hands the worker an Arrow batch instead of a row, so the crossing happens once per batch.

The rule from the Polars half still holds, only more so: look for a builtin first.



One core, four libraries: SQL, streaming, MLlib, GraphX



All four sit on the Spark core, whose abstraction is the resilient distributed dataset (RDD). The core also handles task dispatch, scheduling and I/O.

They share one API and one deployment, in Scala, Java, Python, R and SQL with almost identical names. Training a model on a batch and then scoring a stream needs no second system.



Batch and streaming share the same transformations

Batch: aggregate a database table

```
df = (spark
    .read
    .format("jdbc")
    .option("url", url)
    .option("dbtable", "people")
    .load()
)

counts_by_age = df.groupBy("age").count()
```

Streaming: count words as they arrive

```
lines = (spark
    .readStream
    .format("socket")
    .option("host", host)
    .option("port", port)
    .load()
)

words = lines.select(
    explode(split(lines.value, " ")).alias("word"))
word_counts = words.groupBy("word").count()

query = (word_counts.writeStream
    .format("console").start())
```

Swap `read` for `readStream` and the business logic is identical: `select`, `groupBy`, `count`.



SQL and the DataFrame API compile to the same plan

Both go through the same optimiser and run on the same executors. Neither is faster, and neither is "more distributed" than the other, so the choice is about maintaining the code.

Write it in SQL

```
with open("min_max_by_user.sql") as fh:
    query = fh.read()
spark.sql(query)
```

- Everyone on the team reads it
- Says what, not how
- No second API to learn

Write it in Python

- Functions and modules instead of copy and paste
- Branches and loops
- Unit tests on the transformation
- Type checking and linting before you deploy
- Machine learning in the same program

Spark distributes the work either way, so scale is not the tiebreaker.



Running it in practice



Polars ships as a **wheel**; PySpark needs a **JVM**

Polars

```
uv add polars
```

- One wheel with a Rust binary inside
- No runtime to install, no services to start
- The same code runs in a notebook, a Lambda and a CI job

PySpark

```
uv add pyspark          # ~300 MB  
apt install openjdk-17-jre # + JAVA_HOME
```

```
spark = (SparkSession.builder  
    .master("local[*]")          # one machine  
    .config("spark.sql.shuffle.partitions", 200)  
    .getOrCreate())
```

- `local[*]` for your laptop, a cluster manager for anything else
- Settings live in `conf/spark-defaults.conf`, this repo ships one

The cost of the second engine is the runtime around it: a JVM of the right version, a cluster manager, and configuration.



Connectors need matching JARs and configuration

AWS S3 outside EMR

```
config = {
    "spark.jars.packages":
        "org.apache.hadoop:hadoop-aws:<version>",
    "spark.hadoop.fs.s3a.aws.credentials.provider":
        "org.apache.hadoop.fs.s3a"
        ".SimpleAWSCredentialsProvider",
}
conf = SparkConf().setAll(config.items())
spark = SparkSession.builder.config(
    conf=conf).getOrCreate()

df = spark.read.csv(f"s3a://{BUCKET}/{KEY}",
                    header=True)
```

Snowflake

```
snowflake_pkgs = [
    "net.snowflake:snowflake-jdbc:<version>",
    "net.snowflake:spark-snowflake_2.12:<version>",
]
config = {
    "spark.jars.packages": ",".join(snowflake_pkgs),
}
conf = SparkConf().setAll(config.items())
spark = SparkSession.builder.config(
    conf=conf).getOrCreate()
```

Versions have to line up: `hadoop-aws` against the Hadoop libraries in your Spark install, `spark-snowflake` against your Spark version. Then come the authentication options. Polars reads `s3://` with `pip install polars[aws]`.



Readers and writers handle formats, folders and partitioning

```
frame = spark.read.csv(
    str(csv_file_path), sep=";", header=True)

frame = (
    spark.read
    .options(header="true", sep=";", inferSchema=True)
    .format("csv")
    .load(str(csv_file_path))
)

frame.write.parquet(
    "some_location",
    mode="overwrite",
    partitionBy="country",
)
```

Hive-partitioned output

```
├─ date=2021-10-08
│   ├── train_id=200
│   │   ├── part-00000-6d...
│   │   └── part-00001-6d...
│   ├── train_id=231
│   │   ├── part-00000-6d...
│   │   └── part-00001-6d...
│   └── train_id=4361
│       ├── part-00000-6d...
│       └── part-00001-6d...
├─ date=2021-10-09
│   └── train_id=200
```

One file per partition per writer, in folders named after the column value. The same layout Polars read from earlier: partition on what you filter on first.



When to reach for Spark

Spark earns its keep

- The data does not fit one machine, and streaming it through does not help
- You already run a platform that hosts it: Databricks, EMR, Kubernetes
- You need batch, streaming and ML behind one API
- A job runs long enough that losing a node matters

Stay on one node

- It fits in memory, or streams through a single machine
- You want a fast local loop and a short install
- The team is small and the pipeline is simple
- Cost per query matters more than peak scale

A cluster costs a JVM, configuration, shuffles and idle CPUs, whether or not the data needs the scale. The decision tree from the first half still applies.



What we covered

1. **Same data, different storage.** Operational and analytical systems hold the same facts. Storing them row by row or column by column is what decides which questions are cheap to ask.
2. **Let the engine do the heavy lifting.**
 - It turns what you asked for into a plan, then reorders joins and pushes filters and column selections down into the scan, so it reads less.
 - You help it by storing data columnar and partitioning along the questions you actually ask. It can only skip what the layout allows.
3. **Relational algebra has lasted.** Project, filter, join, aggregate, window. SQL and the DataFrame API are two ways of writing the same operations.
4. **One node goes a long way.** For most pipelines a single machine is simpler, cheaper and quicker to iterate on. Reach for Spark when the data is genuinely large.





Thank you

